

The silent bug audit: catch *type errors* before they poison your analysis

Types are invisible until they break something. This worksheet forces you to see what Python actually stores — not what you assumed it stored.

NAME

DATE

SESSION

1 Your last analysis — type audit

- Pick a recent script or notebook. List 3 variables you used. For each, write: **(a)** what type you assumed it was, **(b)** what `type()` actually returns.
- Did any column arrive as strings when you expected numbers? How did you find out — error, wrong output, or *never?*
- Write one line where a silent concatenation bug could have hit your work. Example: `"Total: " + str(count)` vs `"Total: " + count`

VARIABLES & ACTUAL TYPES

2 Lists vs dicts — what are you actually storing?

- You have employee records with name, salary, department. Would you use a list of dicts or a flat list of salaries? Write your reasoning in *one* sentence.
- Write a list comprehension that filters salaries above your own threshold (pick a number). Don't look it up — write it from memory first, then verify.
- When does a list of dicts become the *wrong* tool? Name one scenario where you'd switch to something else.

REASONING & CODE

3 Function refactor — your duplicated code

- Identify one block of code you've copy-pasted across notebooks or scripts. Write the block here (or describe it in plain English).
- Rewrite it as a named function with at least one parameter. Add a *default argument* where it makes sense.
- What would break if someone passed the wrong type into your function? How would you catch it?

```
def my_function(data, threshold=0.5):  
    # your refactored logic here  
    return result
```

ORIGINAL BLOCK & REFACTORED FUNCTION

4 The four primitives — intention check

- For each type below, write one *real* example from your own data work: `int` · `float` · `str` · `bool`
- Where in your code have you mixed `int` and `float` without thinking? What was the result?
- Write the conversion that fixes this: `age_column = ['28', '35', '42']` — you need to sum it.

```
ages = [int(a) for a in age_column]    # your fix here  
total = sum(ages)
```

PRIMITIVES & CONVERSIONS

5 Build your personal type-safety checklist

- List 3 columns in datasets you work with regularly. What type *should* each be? What type does Python often load them as?
- Write a 3-step pre-analysis check you'll run before touching any new CSV.
- After Tutorial 1's average calculation: rewrite it as `calculate_average()`. Then add `is_outlier(value, data, threshold=2.0)` that returns `True` / `False`.

```
def calculate_average(data): ...  
def is_outlier(value, data, threshold=2.0): ...
```

CHECKLIST & FUNCTIONS
